Deep Learning for Natural Language Processing

AIL7390



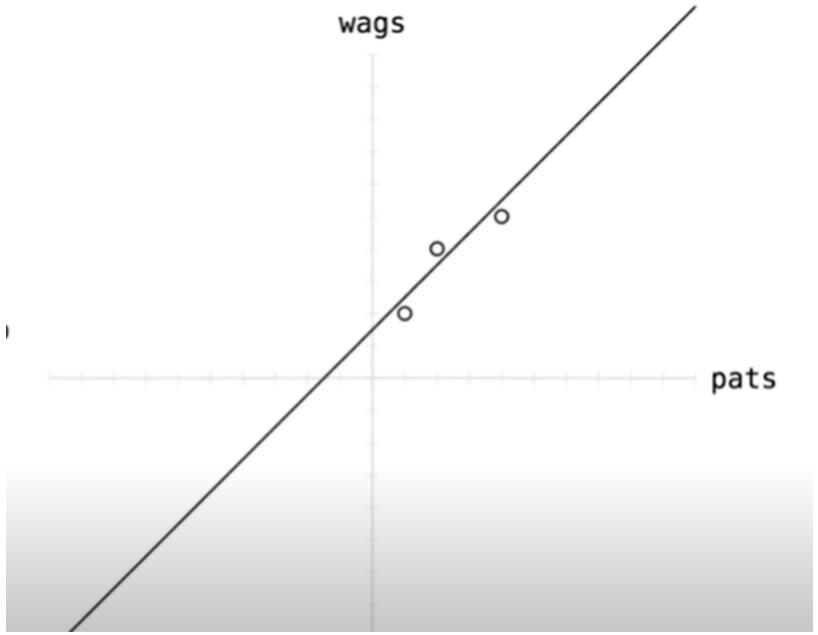
Deeksha Varshney

Content is from various sources

From Regression to Deep Neural Networks

Data	
pats	wags
1	2
2	4
4	5

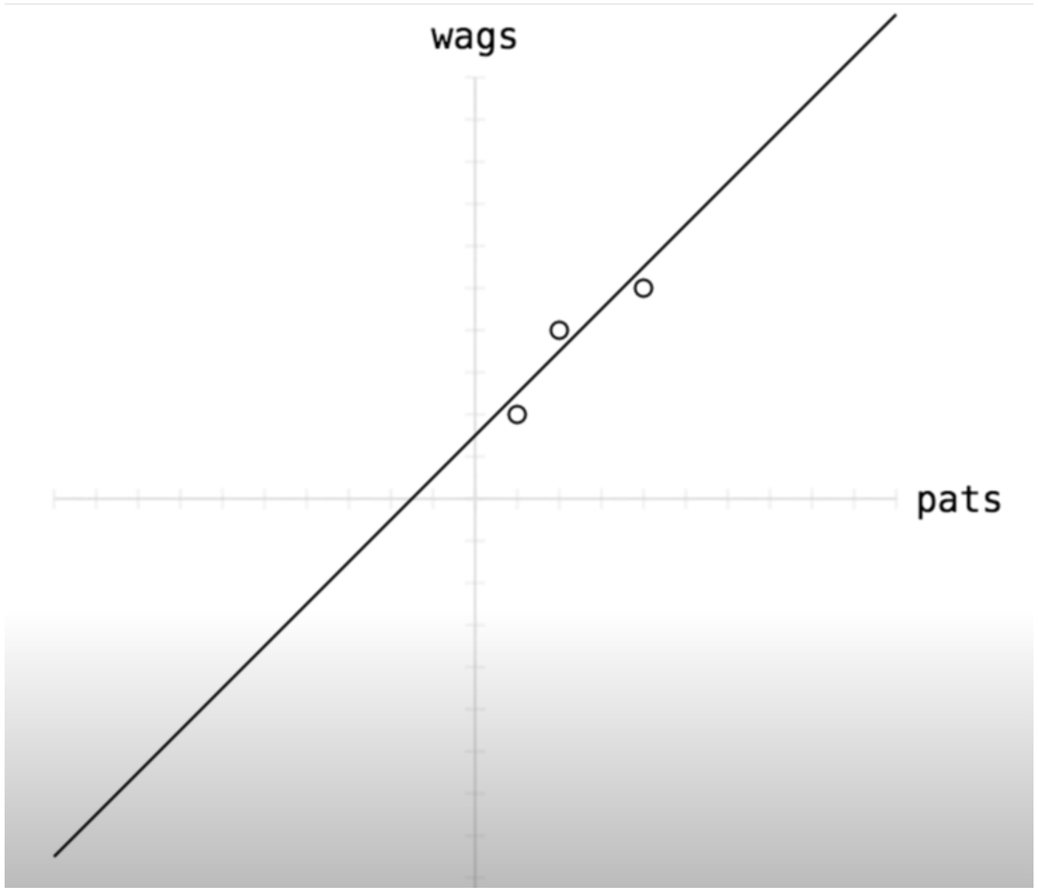
$$wags(pats) = w.pats + b$$



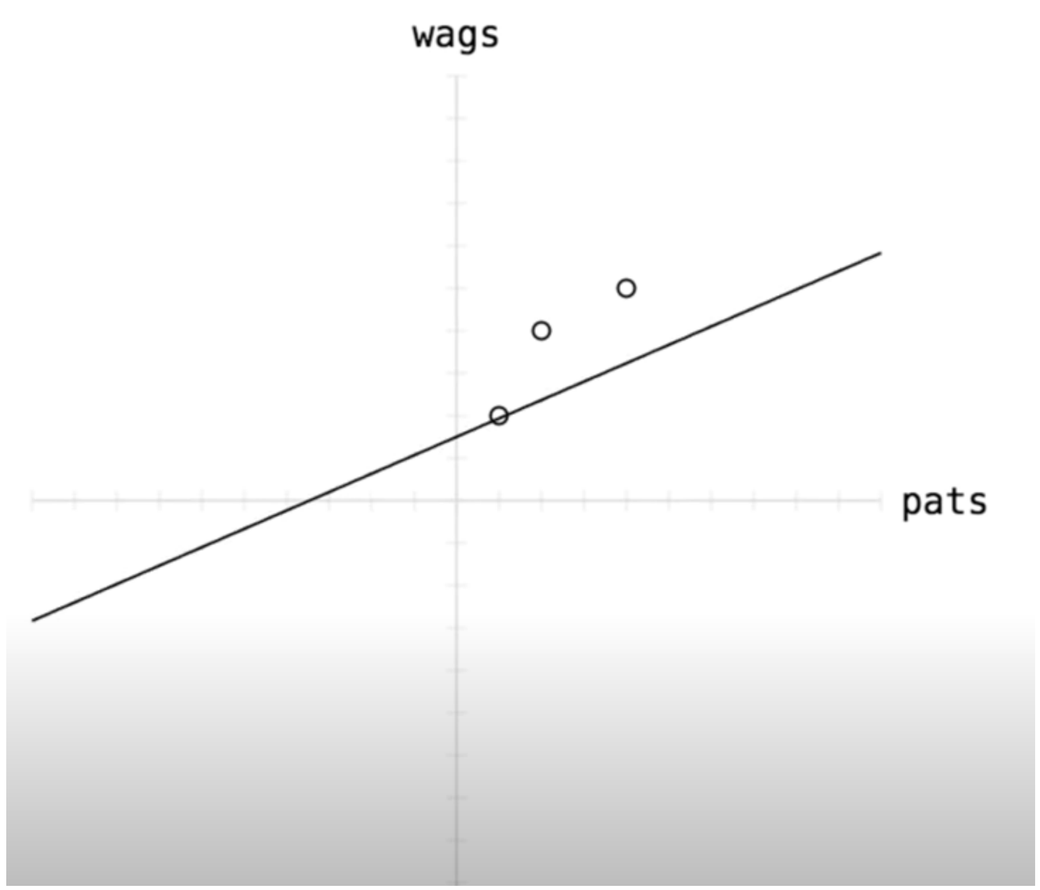
w = 0.25
b = 0.68

pats	wags(pats) (predicte
1	wags(1) → 0.93
2	wags(2) → 1.18
4	wags(4) → 1.68

Linear Regression



w = 1.00
b = 1.50



w = 0.43
b = 1.50

Linear Regression

pats	wags
1	2
2	4
4	5

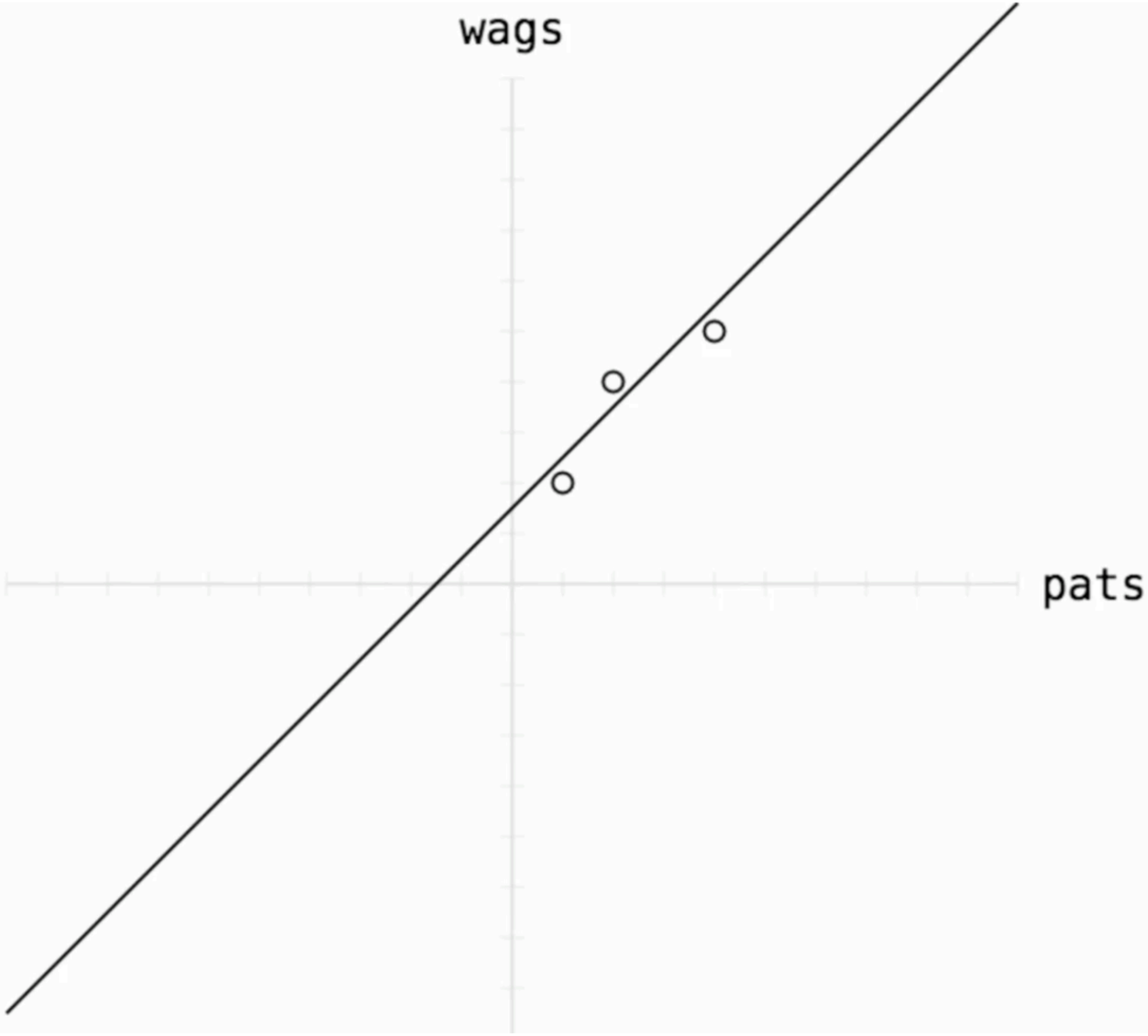
model

wags(pats)=pats•w+b

parameters

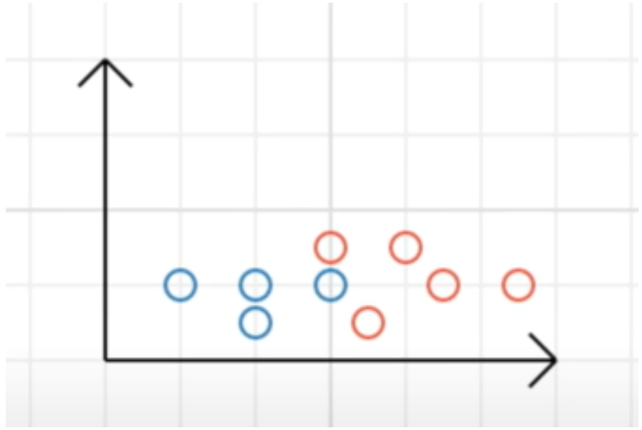
w = 1.00

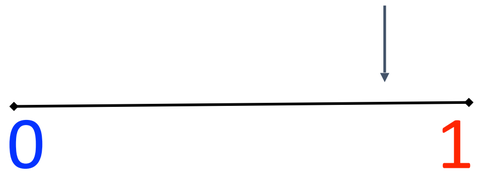
b = 1.50

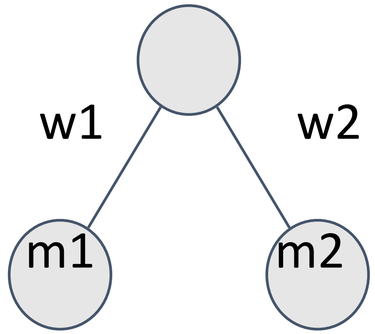


Neural Network

color	🔴	🔵	🔴	🔵	🔴	🔵	🔴	🔵	???
length	3	2	4	3	3.5	2	5.5	1	4.5
width	1.5	1	1.5	1	0.5	0.5	1	1	1







NN(m1,m2) = sigmoid (w1.m1 + w2.m2 + b)

Logistic Regression

Sigmoid maps any real number to **(0, 1)**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Why this matters:

- Linear models output values from $-\infty$ to $+\infty$
- For binary classes, we need a **bounded output**
- Values in (0,1) can be interpreted as **probabilities**

Example:

- Output = 0.92 → "very likely red"
- Output = 0.08 → "very likely blue"

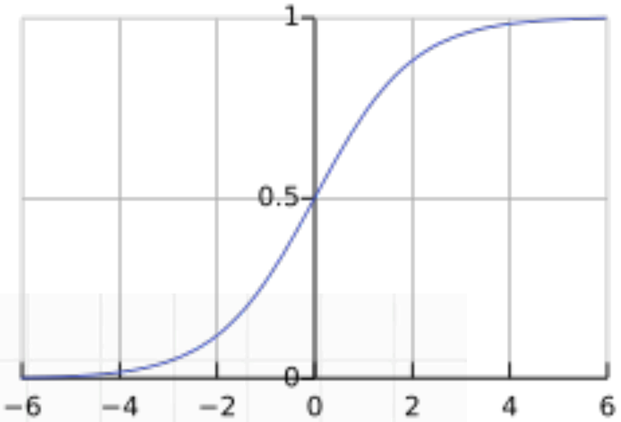
With sigmoid, we can define a simple threshold:

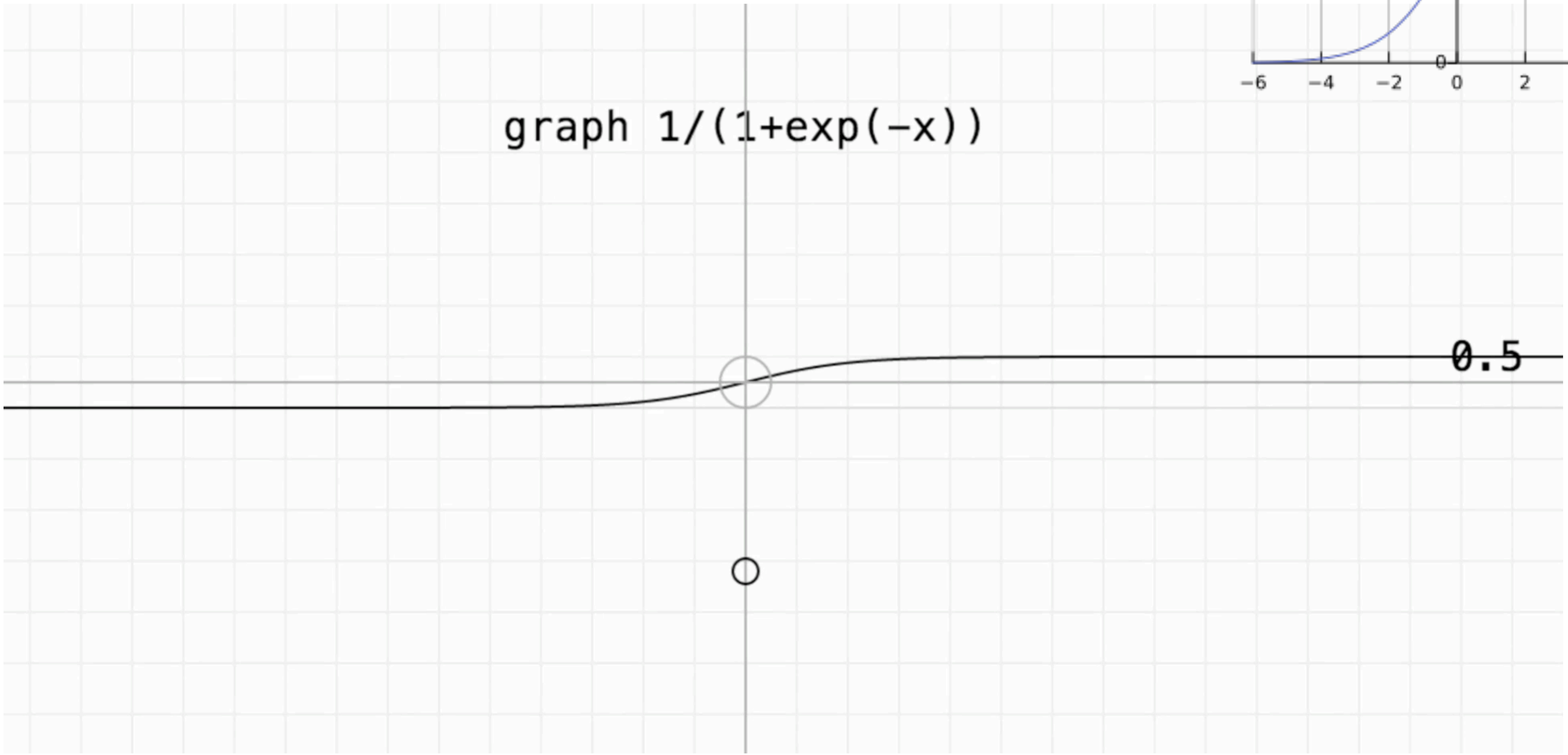
- Output < 0.5 → class 0
- Output ≥ 0.5 → class 1

Without sigmoid:

- There is **no natural cutoff**
- A value like 12 or −7 has no probabilistic meaning

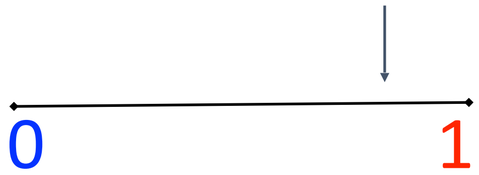
Logistic Regression





Logistic Regression

NN (2,1) = sigmoid (0.5*2 + 0.2 * 1 + 0.3)
= 0.82



Neural Network

"Yes, in this simple example we can see that setting b=4 gives zero cost.
But in real problems, we don't know the answer ahead of time.
So instead of jumping directly to the solution, we'll pretend we don't know it and learn how to reach it using only the information the cost function gives us."

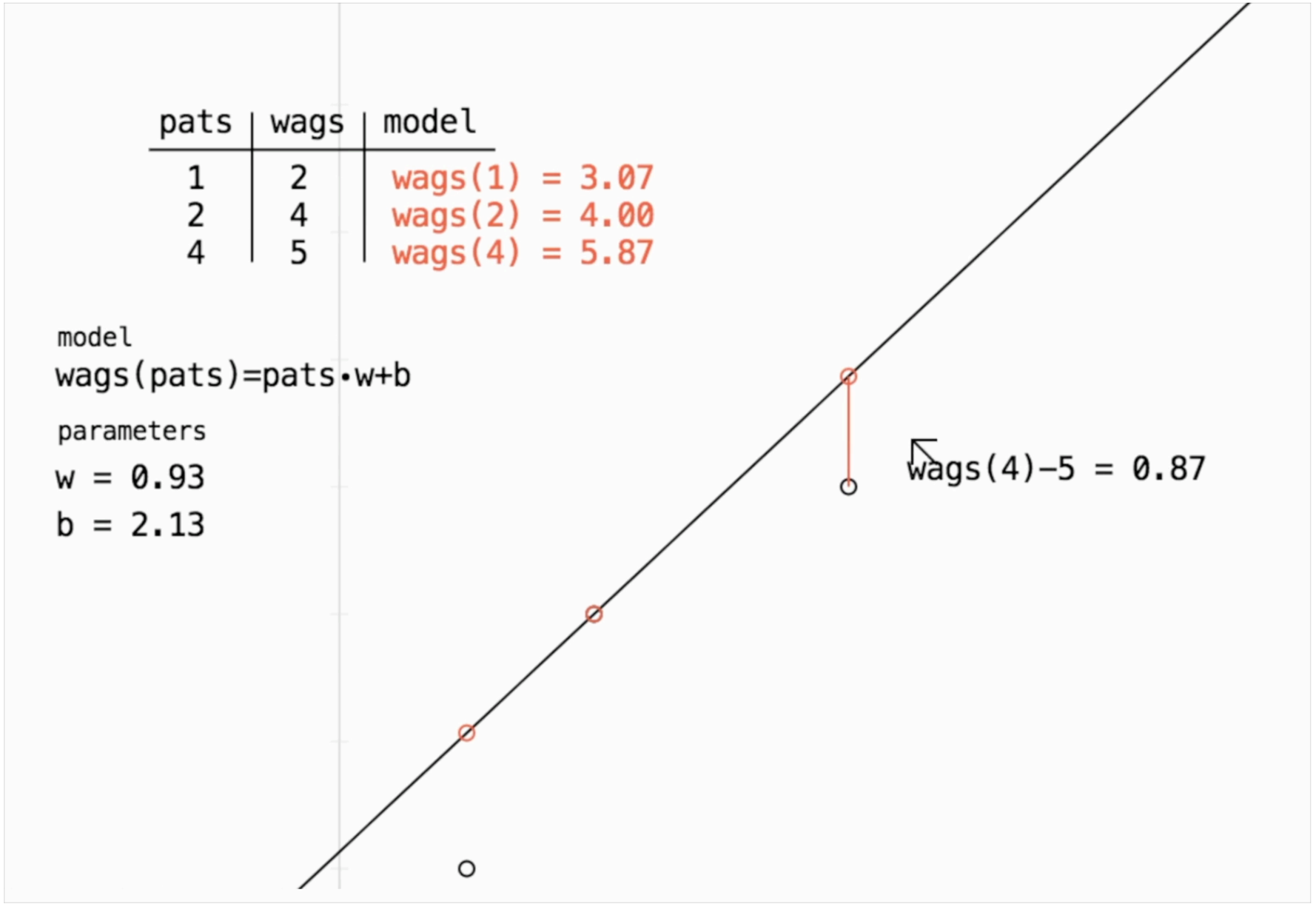
NN() = b

Cost function = (prediction - target)^2 = (b-4)^2

Gradient descent does not find the minimum by knowing the answer , it finds it by following the slope.

A neural network with no inputs is not useful as a model of data , it's a teaching abstraction.
It *technically* fits the mathematical definition of a neural network, but it's introduced **only to explain cost functions and gradient descent**, not to model the world.

Neural Network



Neural Network

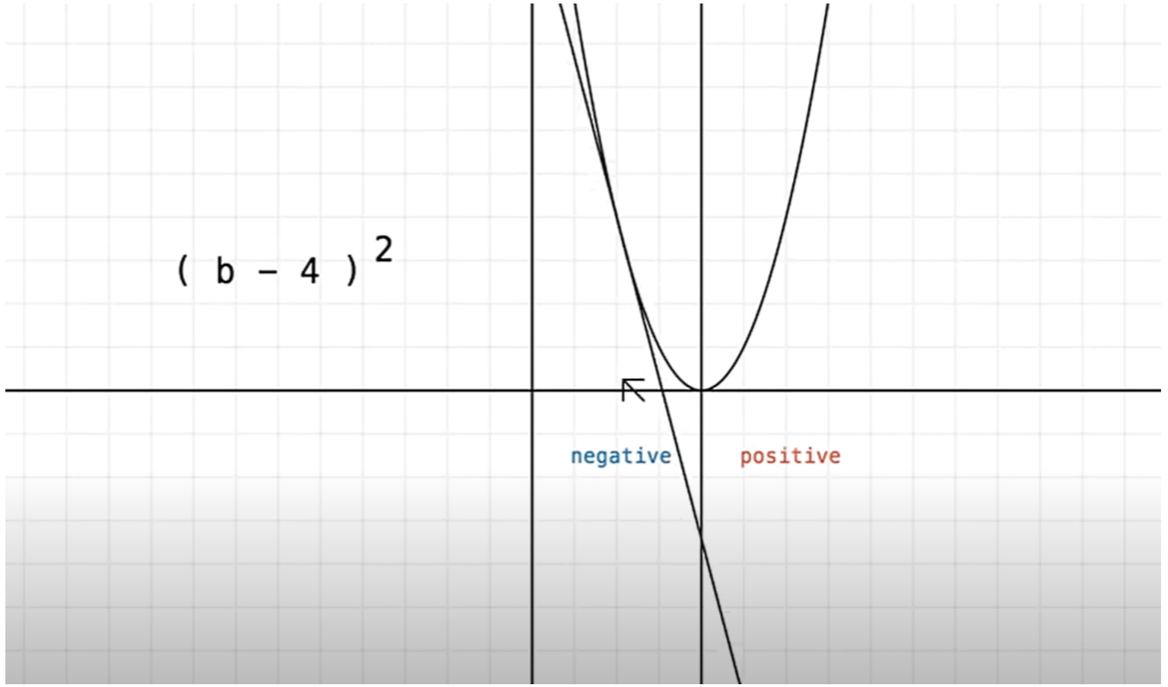
Cost function = (prediction - target)^2 = (b-4)^2

Visualizing the Cost Function

- Horizontal axis: b
- Vertical axis: cost

This forms a **parabola**:

- Minimum at b=4
- Cost increases as we move away



Neural Network

How do we know **which direction** to change or **what should be the value of b**?

Increase it? Decrease it?

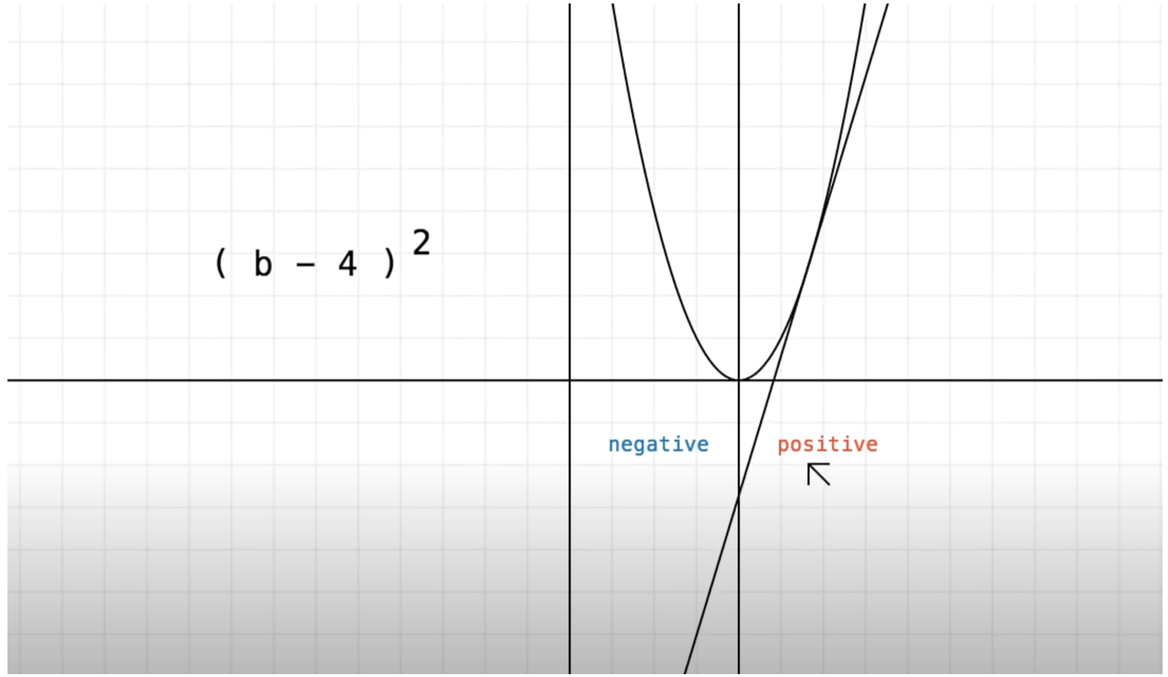
Answer: Look at the **slope** of the cost function

Slope Tells Direction

- Slope > 0 → decrease or **when we want to decrease b we have a positive slope**
- Slope < 0 → increase or **when we want to increase b we have a negative slope**
- Slope = 0 → at minimum

Slope tells us:

- Direction to move
- How aggressively to move



Neural Network

Gradient Descent Update Rule

Update rule:

$$b \leftarrow b - \alpha \frac{dC}{db}$$

Where:

α = learning rate

$\frac{dC}{db}$ = slope (derivative)

Learning Rate Intuition

- Too large → overshoot, oscillate
- Too small → very slow learning

We use a **fraction of the slope** to stay stable.

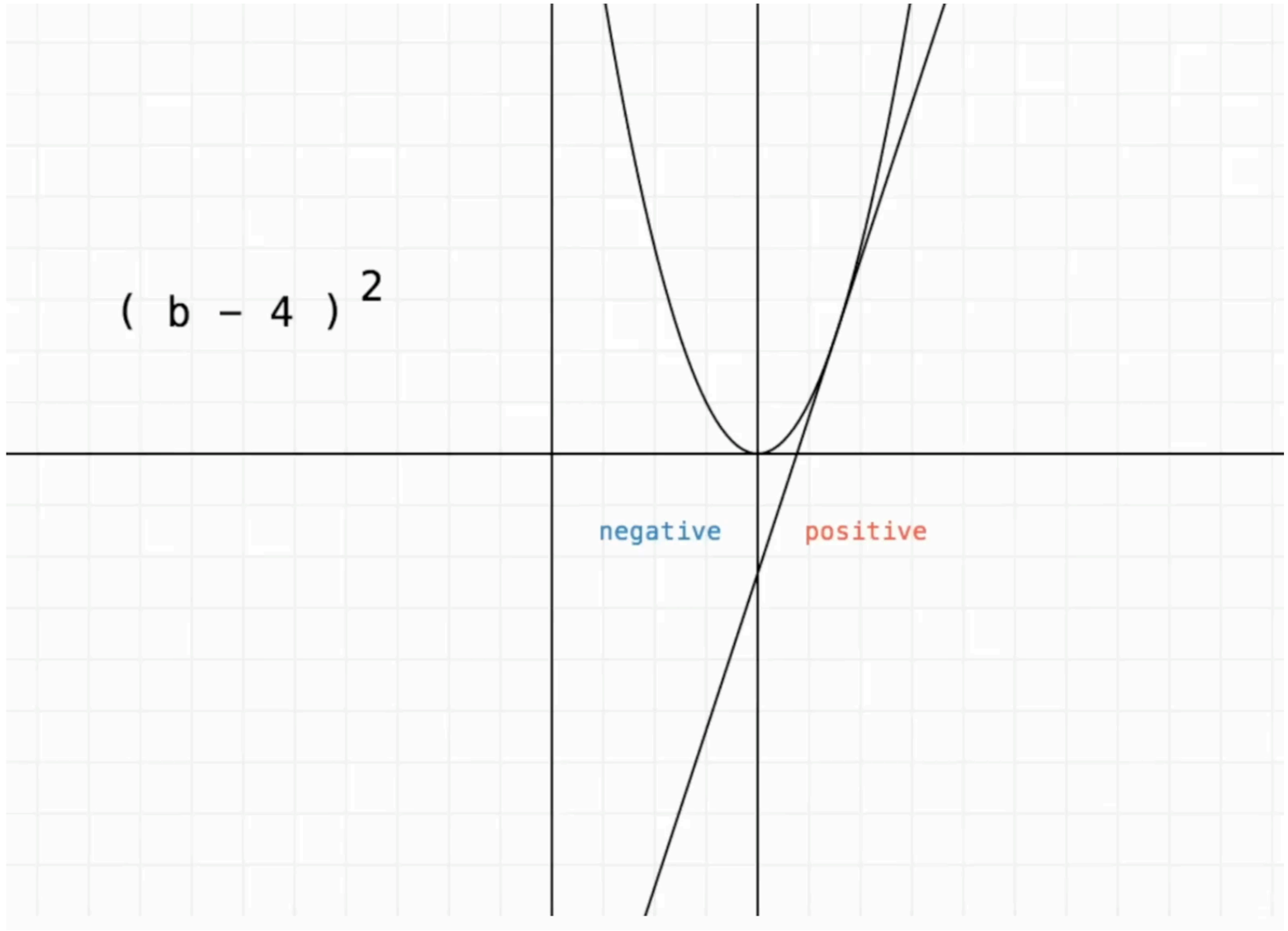
Why Derivatives Matter

- Derivative = exact slope
- No guessing
- Works for complex models

Derivative of the Cost Function

D cost(b) / db = 2(b-4)

Neural Network



pats	wags	model	error
1	2	wags(1) = 2.37	c1= (wags(1) -2)^2 = 0.13
2	4	wags(1) = 3.37	c2 = (wags(2) - 4)^2 = 0.40
4	5	wags(1) = 5.37	c3 = (wags(4) - 5)^2 = 0.13
			Cost = c1+ c2 + c3 = 0.67

model

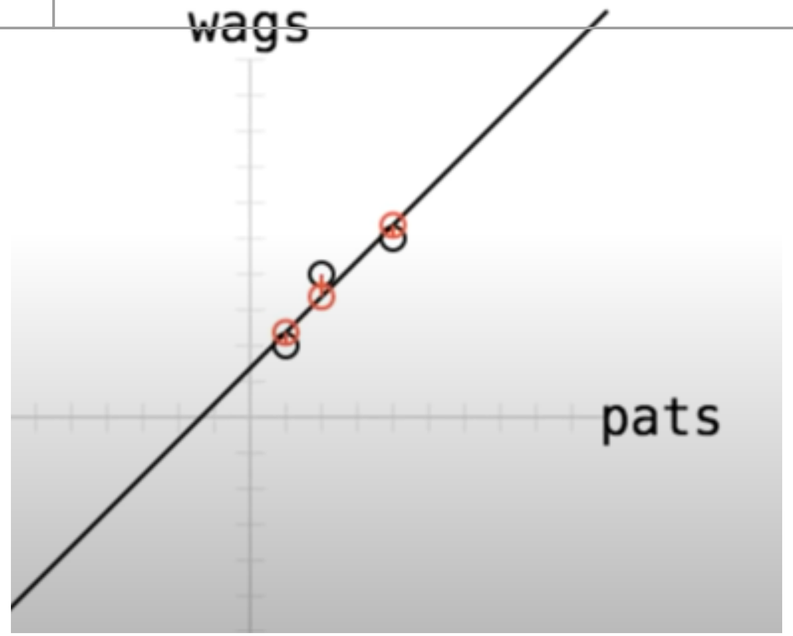
wags(pats) = pats.w + b

Parameters

w = 1.00

b = 1.37

$$\text{Cost} = \sum_{i=1}^{m} (\text{model}(x_i) - \text{target}_i)^2$$



Neural Network

We previously defined the cost:

$$C(w,b) = (w \cdot 1 + b - 2)^2 + (w \cdot 2 + b - 4)^2 + (w \cdot 4 + b - 5)^2$$

- Inputs: model parameters w,b
- Output: a single number (cost)

This is now a **function of two variables**.

Neural Network

From Curve to Surface

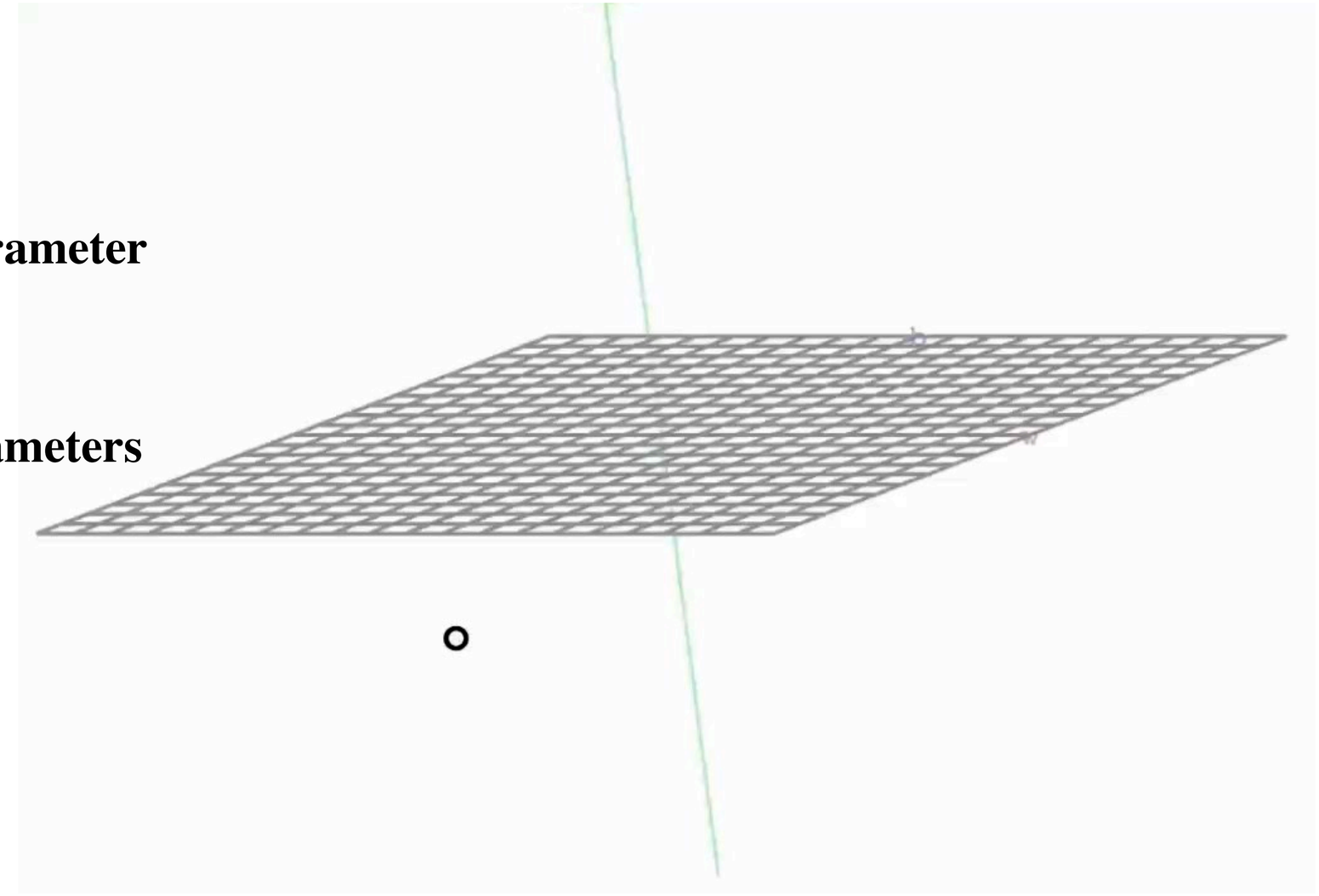
Before:

- Cost depended on **one parameter**
- Visualization: a curve

Now:

- Cost depends on **two parameters**
- Visualization: a **surface**

We need one more axis.



Neural Network

1. 3D Coordinate System

We define:

- x-axis → w
- z-axis → b
- y-axis → cost

Each (w,b) pair maps to a height C(w,b).

2. Plotting a Single Point

Pick a random: w, b

Evaluate cost and plot: (w,C(w,b),b)

3. The Bowl Emerges

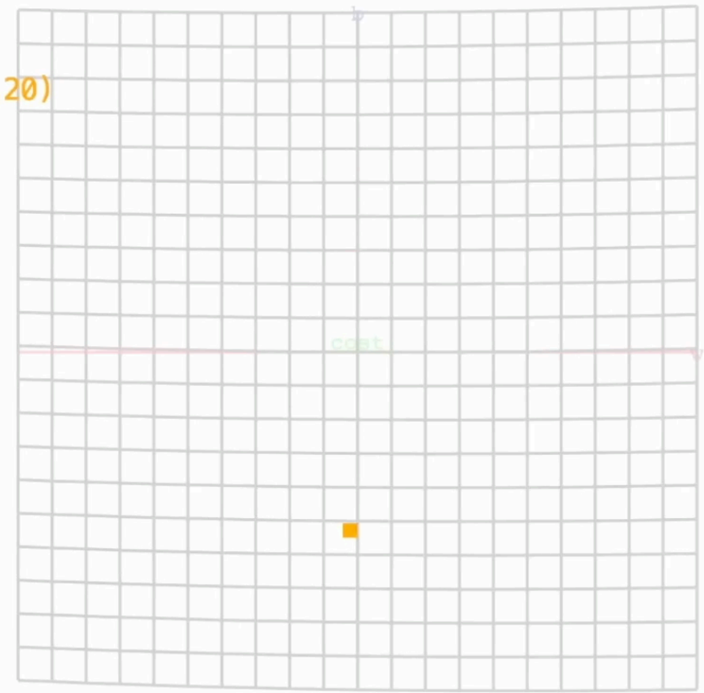
When cost becomes the vertical axis:

- The surface forms a **bowl**
- One clear lowest point

Neural Network

$$cost(w,b) = (w \cdot 1 + b - 2)^2 + (w \cdot 2 + b - 4)^2 + (w \cdot 4 + b - 5)^2$$

```
surface(c(w,b)=h·cost(w,b))
h = 0.00
w = -0.22
b = -5.28
scatter([[w,h·cost(w,b),b]],20)
```



Optimization Goal

Our goal:

- Start anywhere on the bowl
- Move downhill
- Reach the bottom

This is **gradient descent** in 2D.

Neural Network

Why Partial Derivatives?

Cost depends on:

- w
- b

We ask:

- How does cost change if we change **only w**?
- How does cost change if we change **only b**?

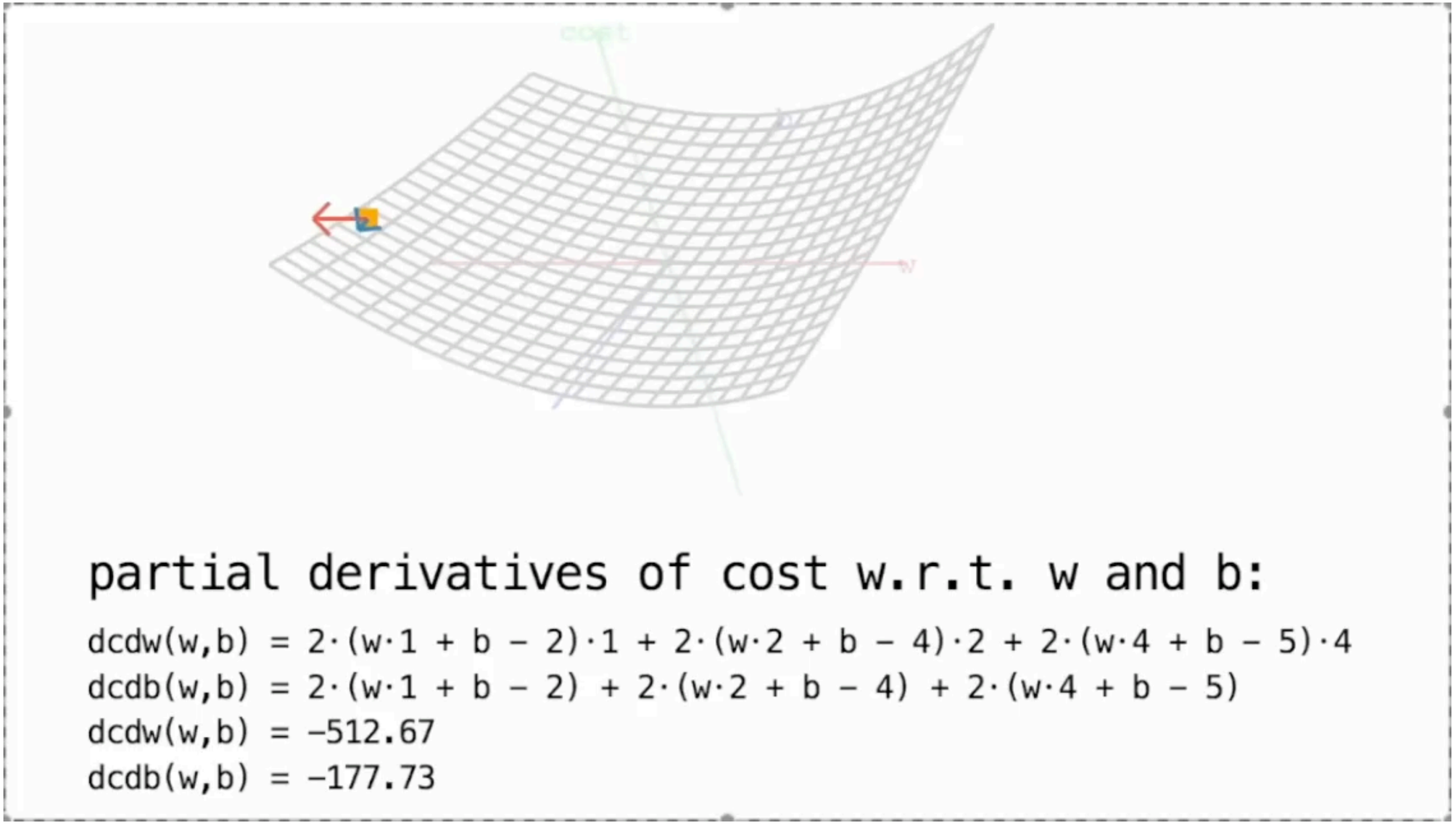
Partial Derivative w.r.t. w : $\partial C\,(w,b)/\partial w$

- Treat b as constant
- Slope in the w-direction
- Vis: Slice of the bowl along the w-axis

Partial Derivative w.r.t. b : $\partial C\,(w,b)/\partial b$

- Treat w as constant
- Slope in the b-direction
- Vis: Slice of the bowl along the b-axis

Neural Network



The visualization is created using Wolfram Mathematica, leveraging its 3D surface plots and interactive parameter controls.

Non-linearities, old and new

Threshold Units

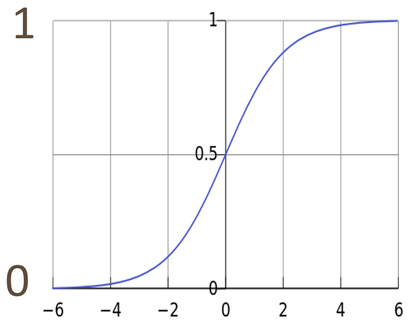
- Output: 0 or 1
- Problem: **no gradient** → no learning

Nonlinearities:

- Enable function approximation
- Enable gradient-based learning

logistic ("sigmoid")

$$f(z) = \frac{1}{1 + \exp(-z)}$$



Smooth, probabilistic interpretation

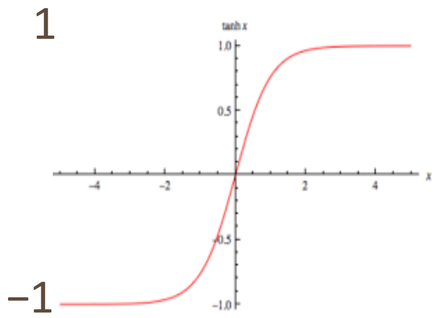
Problems:

- Saturation
- Outputs always positive

tanh

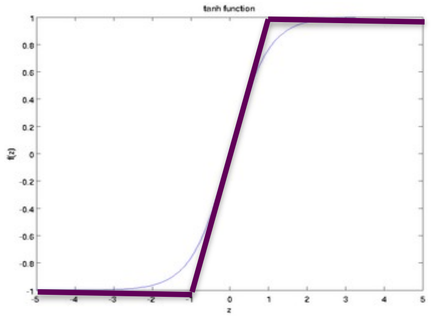
- Zero-centered
- Scaled sigmoid
- Common in older neural networks

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



hard tanh

$$\text{HardTanh}(x) = \begin{cases} -1 & \text{if } x < -1 \\ x & \text{if } -1 <= x <= 1 \\ 1 & \text{if } x > 1 \end{cases}$$



tanh is just a rescaled and shifted sigmoid (2×as steep, [−1,1]):

$\tanh(z) = 2\text{logistic}(2z) - 1$, The factor 2z makes tanh twice as steep at 0

Logistic and tanh are still used (e.g., logistic to get a probability)

Sigmoid vs Tanh vs Hard Tanh

- Sigmoid:
  - Outputs always positive
  - Can push activations to be biased upward
- Tanh:
  - Zero-centered
  - Often leads to **faster convergence**

This is why tanh was historically preferred in hidden layers.

Problem with tanh

- Requires **exponentials**
- Slower to compute (especially on older hardware)

Idea

“Can we keep tanh’s behavior, but remove expensive math?”

Answer:

- Replace the curved parts with **straight lines**

Non-linearities, old and new

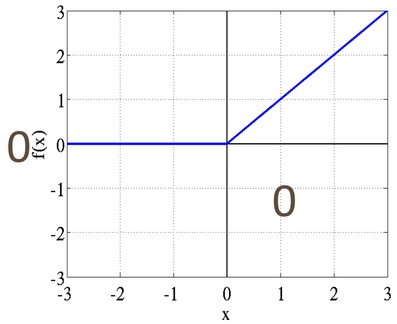
However, now, for deep networks, the first thing to try is ReLU: it trains quickly and performs well due to good gradient backflow.

"ReLU variants solved gradient flow problems, but modern models especially transformers need smoother behavior."

proposals mitigate GELU is frequently used with Transformers (BERT, RoBERTa, etc.)

(Rectified Linear Unit)
ReLU

$$\text{ReLU}(z) = \max(z, 0)$$



What is the gradient when x<0?

Answer:

- **0**

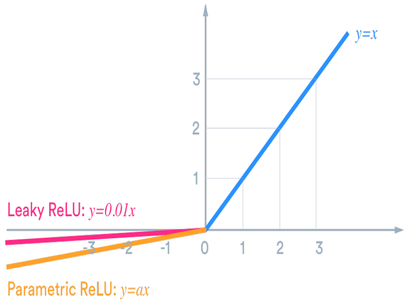
Pros:

- Simple
- Strong gradients when active

Cons:

- "Dead" neurons for negative inputs

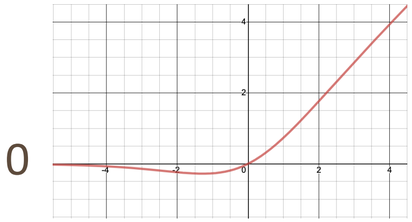
Leaky ReLU / Parametric ReLU



"What if activation strength changed *smoothly*, not abruptly?"

Swish arXiv:1710.05941

$$\text{swish}(x) = x \cdot \text{logistic}(x)$$

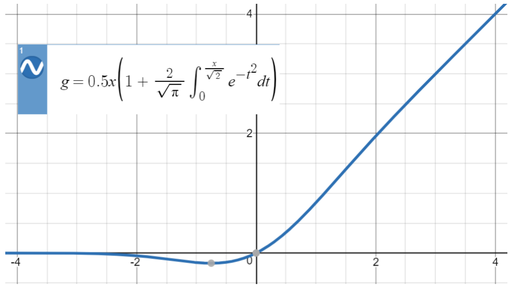


Multiply input by a **soft gate**
Gate ranges between 0 and 1
Allows:

- small negative values

GELU arXiv:1606.08415

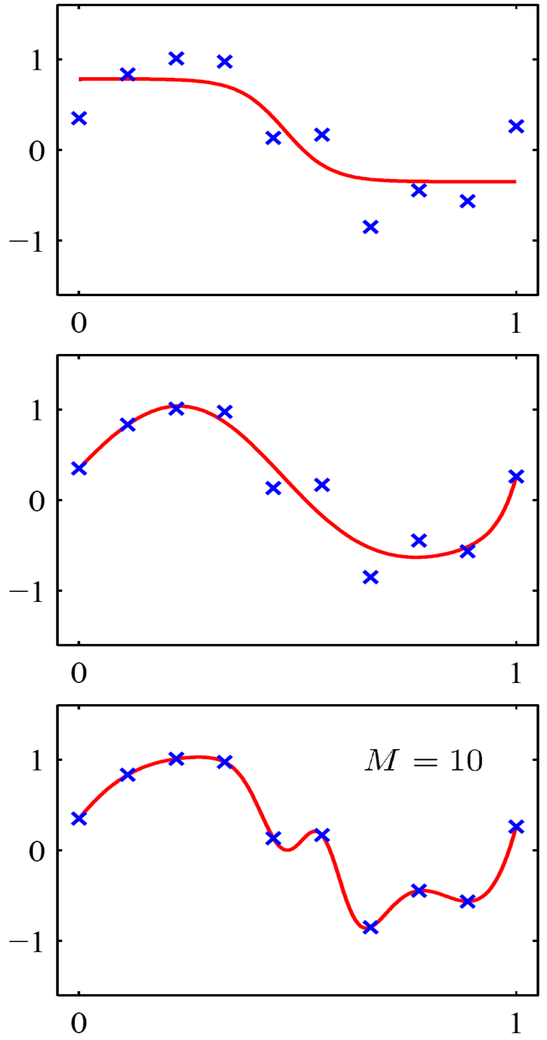
$$\text{GELU}(x) = x \cdot 2P(X \le x), X \sim N(0,1) \approx x \cdot \text{2logistic}(1.702x)$$



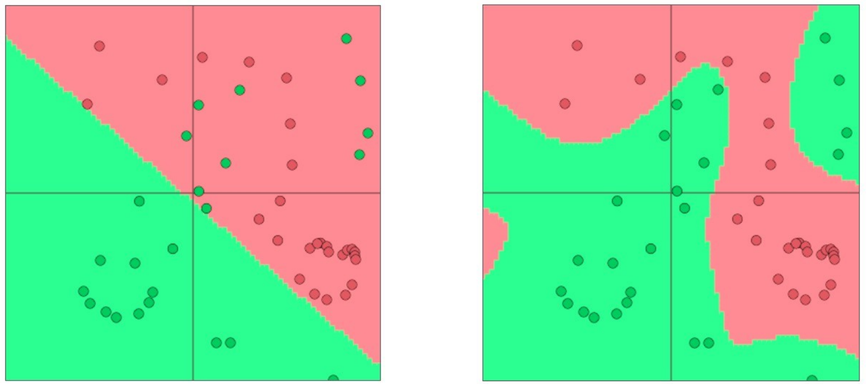
- P(X≤x) is the **Gaussian CDF**
- It tells us *how likely x is to be positive*
- GELU **scales x by that probability**

Non-linearities (i.e., "*f*" on previous slide): Why they're needed

- "What do we use neural networks for?"
  - regression
  - classification
  - mapping inputs → outputs
- "At the end of the day, a neural network is just a **function approximator**."
- Without non-linearities, deep neural networks can't do anything more than a linear transform
- Extra layers could just be compiled down into a single linear transform: $W_1 W_2 x = Wx$.
  "Stacking linear layers doesn't make the function more powerful - it just wastes compute."
- But, with more layers that include non-linearities, they can approximate any complex function!



"Linear layers stretch and rotate space.
Non-linearities **bend** it."



Non-linearities are necessary, but they also introduce problems.
More non-linear layers = more expressive power **and** harder optimization; over-fitting issues.